

Crinava: The Definitive Ultra-Master Blueprint

[!IMPORTANT]

To any AI or Developer reading this: This document contains the molecular-level breakdown of the Crinava repository. It is explicitly designed to leave zero ambiguity about how the state is managed, how the UI is laid out, and how complex data (like live tug-of-war debates, JSONBin syncing, and Cricsheet scorecard parsing) is handled.

Table of Contents

- 1. Core System Architecture & Technologies
 - 2. Global Navigation System (ORBITAL_OS)
 - 3. The Debate Arena & Physics Engine (`App.tsx`)
 - 4. The Matches Section & Data Parsing (`MatchesSection.tsx`)
 - 5. The Prediction Game Engine (`PredictionGame.tsx`)
 - 6. Command Deck & Wallet Integrations (`AuthModal.tsx`, `RazorpayCheckout.tsx`)
 - 7. Database Schemas & AI Services (`supabaseClient.ts`, `ai.ts`)
 - 8. Master Bug Resolution History
-

1. Core System Architecture & Technologies

Crinava is built as a single-page React application optimized for rapid visual state updates using Framer Motion.

The Stack:

- **Core:** React 19.0.0, Vite 6.4.3, TypeScript 5.8.2.
 - **Styling:** Tailwind CSS v4 with bespoke configuration for the dark-mode futuristic sports aesthetic (featuring colors like `metallic-gold`, `bg-primary`, and `accent-default`).
 - **Animations:** `motion/react` (Framer Motion v12) handling complex SVG path interpolations.
 - **Database & Auth:** `@supabase/supabase-js` v2.100.1 connecting to a PostgreSQL instance with Row-Level Security. Also integrates CockroachDB endpoints (`/api/series`).
 - **State Management:** `zustand` for cross-component global state, supplemented by `localStorage` for ephemeral game sessions.
 - **AI & Integrations:** `@google/genai` (Gemini 1.5/2.0 API integrations), `openai` (for embeddings and fallback generation), `razorpay` (for real-money top-ups).
-

2. Global Navigation System (ORBITAL_OS)

The application uses an architectural shell known as **ORBITAL_OS**. This shell completely surrounds the core routing views, ensuring that navigation is ubiquitous.

Top Header (`header` tag in `App.tsx`)

- **Left Hamburger Toggle:** Triggers a state (`isProfileOpen`) which conditionally mounts the **Command Deck** (an off-canvas sidebar menu).
- **Center Brand:** Displays `ORBITAL_OS` or `CRINAVA` depending on the view.
- **Right Icon:** A notification bell or quick-status toggle.

Bottom Navigation Bar

- A fixed `bottom=0` glassmorphic navigation bar housing tabs: HOME, MATCHES, RAFFLE, STORE, BLOG, and DEBATE.
- When the user selects DEBATE, the main `<main>` container mounts the Debate Arena. When selecting STORE, it mounts the wallet UI.

[!TIP] The bottom navigation bar is conditionally hidden when the user enters the `PredictionGame.tsx` module, which injects its own contextual sub-navigation (`PREDICT`, `VERDICT`, `MOMENTUM`, etc.).

3. The Debate Arena & Physics Engine

(`App.tsx`)

This is the most technically complex visual component in the application. It maps public sentiment (votes) to a physical Tug-of-War SVG animation.

State Variables

- `debates`: Array fetched from Supabase storing topics (e.g., *Virat Kohli vs Rohit Sharma*).
- `isTrafficSimulating`: Boolean toggle for the live traffic mode.
- `simBiasesRef`: A `useRef<Record<string, number>>` mapping each debate ID to an independent mathematical momentum bias (between `-0.85` and `0.85`).

The Mathematical Momentum Engine

When `isTrafficSimulating` is true, an interval fires every 200ms:

1. **Bias Drift**: Every 1.5 seconds, the `biasDriftIntervalRef` randomly nudges the bias for each debate card. If bias is `+0.6`, the left side has a 60% higher chance of receiving a burst of votes.

2. Vote Injection & Square Root Scaling:

```
const totalVotes = currentDebate.votes_for + currentD
const baseIncrement = 5 + Math.floor(Math.sqrt(totalV
```

Why this exists: If a debate has 20,000 votes, adding 5 votes will not visibly move the UI percentage. By scaling the increment using `Math.sqrt`, the UI maintains highly volatile momentum swings regardless of how massive the total vote count gets.

Framer Motion SVG Rendering

The characters (Red and Blue) are rendered in an SVG with `viewBox="0 0 400 192"`.

- **The Lean Calculation:** `lean = (percentage - 50) * 0.8`. If Red is winning 70% to 30%, `lean = 16`.
 - **The (0,0) Anomaly Fix:** Conditional rendering (`{wins && <Crown>}`) caused Framer Motion to glitch and fly elements in from coordinates `(0, 0)`. **The Fix:** All visual elements are permanently in the DOM. We animate their `opacity: wins ? 1 : 0` and pass `initial={false}` to the motion tags.
 - **The Rope:** Instead of clipping paths, the rope is drawn as two distinct `<motion.path>` elements that meet exactly at `(200 + lean, 90)`, eliminating browser rendering glitches.
-

4. The Matches Section & Data Parsing (MatchesSection.tsx)

A massive 128KB monolithic component responsible for rendering live matches and historical scorecard data.

Cricsheet JSON Parsing (`parseScorecard`)

The `MatchesSection` includes a highly advanced helper function `parseScorecard(rawInfo)`.

- **Input:** It takes raw JSON dumped from Cricsheet or CockroachDB.
- **Logic:** It iterates over every single delivery (`over.deliveries`) in an innings.
- **Batters:** It tracks runs, balls, fours, sixes, and builds a dynamic strike rate ($sr = (runs/balls) * 100$). It calculates dismissals (caught and bowled, lbw b bowler).
- **Bowlers:** It calculates legal balls, wides, no-balls, tracks dot balls, and computes the economy rate.
- **Output:** Returns a fully structured array: `[{ team, totalRuns, totalWickets, overs, batters, bowlers, extras, didNotBat }]`.

Fallback MVP Calculation

If AI-driven Impact Points are unavailable, the system uses `calculateFallbackMvp(scorecard)`:

- Batters get +1 per run, +2 per six, -0.1 per ball faced.
- Bowlers get +25 per wicket, +1 per dot ball, -0.5 per run conceded.

UI Structure & Filtering

The `TournamentsList` component inside `MatchesSection`:

- Connects to `/api/series` to fetch CockroachDB entries.
 - Contains a massive search & filter UI hidden behind a `showFilters` toggle.
 - Users can filter by **Gender** (Men/Women), **Format** (T20, ODI, Test, IPL), **Year Range** (2000-2026 dual slider), and an explicit **Major Leagues Only** boolean flag.
-

5. The Prediction Game Engine (PredictionGame.tsx)

The `PredictionGame.tsx` (41KB) handles real-time multiplayer predictions. It uses a bespoke serverless state-syncing architecture via `JSONBin.io`.

State Machine Views

The component uses `useState("splash" | "setup" | "onboard" | "game")`.

1. **Splash:** The intro screen ("CRICARENA").
2. **Setup:** Users input `p1` (Player 1) and `p2` (Player 2) names. They can either **Create Room** (generates a 6-character alphanumeric code) or **Join Room** via a code format `ROOM_CODE:BIN_ID` (e.g., `ABCD12:abc123`).
3. **Onboard:** Players select their "Legend" (Rohit, Virat, or Dhoni) which attaches a custom theme (`t-rohit`, color `#2196f3`), and set a 4-digit security PIN.
4. **Game:** The main prediction dashboard.

JSONBin Sync Architecture

Instead of using Supabase Realtime for these ephemeral mini-games, the system saves the state (`room`, `p1`, `p2`, `preds`, `results`, `profiles`) as a JSON payload to `https://api.jsonbin.io/v3/b` using an API Key.

- `syncSave()`: POSTs new data, receives a `bin ID` (saved as `bid`), or PUTs updates if `bid` exists.
- `syncLoad()`: A `setInterval` runs every 5000ms, fetching `${BIN_URL}/${st.bid}/latest` to silently sync the opponent's moves.

In-Game Predictions

- Users lock in match winner predictions (`handleConfirmPick(matchId, teamId)`). The UI prevents changes once the prediction is locked (`st.preds[matchId][st.me]`).
 - Scores are calculated via `calcScore(player)`, comparing `st.preds` against `st.results`.
-

6. Command Deck & Wallet Integrations

Profile Sidebar (Command Deck)

Triggered from the top-left menu. It displays the user's `CORE BALANCE` (e.g., 500 CRN) and their `INTELLIGENCE` rating (IQ).

- Unauthenticated users see a `GUEST_UXR` badge and an `AUTHENTICATE` button.
- The `AUTHENTICATE` button mounts `AuthModal.tsx`.

Authentication (`AuthModal.tsx` & `UsernameSetup.tsx`)

- `AuthModal.tsx` handles OAuth via `supabase.auth.signInWithOAuth({ provider: "google" })` and email Magic Links.
- Upon first login, if the user's `profiles` row is empty, they are redirected to `UsernameSetup.tsx`.
- `useUsernameCheck.ts`: A custom hook that debounces username inputs and queries the `profiles` table to ensure the chosen handle is unique.

Digital Store (`STORE Route` / `RazorpayCheckout.tsx`)

Users purchase "Crinava Coins".

- The UI presents tiers: `STARTER` (100 coins), `MOST POPULAR` (500 coins).
 - Integrating `RazorpayCheckout.tsx`, the system initializes a `Razorpay options` object with `key_id`, `amount` (in paise), and handles the `handler(response)` callback.
 - On success, the UI hits an endpoint or directly updates the user's row in Supabase: `UPDATE profiles SET coins = coins + purchased WHERE id = user.id`.
-

7. Database Schemas & AI Services

Supabase Architecture (`supabaseClient.ts`)

- **profiles:**
 - Columns: `id` (UUID), `username` (Text), `coins` (Integer), `vip_status` (Boolean), `created_at`.
 - RLS: Policies ensure `SELECT` is public but `UPDATE` is strictly restricted to `auth.uid() = id`.
- **debates:**
 - Columns: `id`, `topic`, `left_name`, `right_name`, `votes_for`, `votes_against`.
- **predictions:**
 - Used for long-term real-money/coin wagering. Columns: `user_id`, `match_id`, `predicted_team`, `wager_amount`, `status` (pending/won/lost).

AI Engines (`ai.ts` & `impactEngine.ts`)

- `ai.ts`: Configures `@google/genai` to synthesize match data. The Oracle uses Monte Carlo probabilistic prompts to predict outcomes based on team momentum.
- `impactEngine.ts`: Ingests player stats and normalizes them into an "Impact DNA" rating from 0 to 100, which feeds the

8. Master Bug Resolution History

Critical knowledge for AI agents modifying this codebase:

1. The Native Sandbox Rule:

- NEVER use Node.js executable scripts or `npm run custom_script` to modify files. The Windows Defender antivirus environment flags them. All edits must be done via direct filesystem tool edits (`multi_replace_file_content`).

2. The 5-Second Debate Overwrite:

- When `fetchDebates()` polled Supabase every 5 seconds, it forcefully overwrote the highly volatile states of the Traffic Simulator.
- *The Fix:* We added a strict `if (!isTrafficSimulating)` guard clause in `App.tsx` so the DB does not interrupt the frontend physics engine.

3. The Percentage Dilution Glitch:

- Small static increments (+5 votes) stopped moving the percentage bar once a debate surpassed 10,000 total votes.
- *The Fix:* We implemented a dynamic `Math.sqrt` scaling equation so that vote increments geometrically increase as the pool gets larger.

4. SVG Clipping Catastrophe:

- Standard `<clipPath>` tags fail to re-render smoothly during 60FPS Framer Motion updates in React 19.
- *The Fix:* Avoid `clipPath` for moving borders. Draw explicit intersecting `<path>` coordinates (d strings) dynamically.

END OF DOCUMENT. You now possess the complete architecture, mathematics, and logic schematic for Crinava.